Results in Engineering 31 (2026) 111270

Contents lists available at ScienceDirect

Results in Engineering

journal homepage: www.sciencedirect.com/journal/results-in-engineering

Research paper

Optimizing arithmetic resource allocation in configurable NTT hardware for area-constrained homomorphic encryption ⋆

Omar S. Sonbul [a], Muhammad Rashid [a,*], Amar Y. Jaffar [a], Mohammed J. Alghamdi [b], Muhammad I. Masud [c], Mohammed Aman [d]

[a] Computer and Network Engineering Department, Umm Al-Qura University, Makkah, 24382, Saudi Arabia
[b] Software Engineering Department, Umm Al-Qura University, Makkah, 24381, Saudi Arabia
[c] Department of Electrical Engineering, College of Engineering, University of Business and Technology, Jeddah, 21361, Saudi Arabia
[d] Department of Industrial Engineering, College of Engineering, University of Business and Technology, Jeddah, 21361, Saudi Arabia

ARTICLE INFO

Keywords:
Post–quantum cryptography
Number theoretic transform
Homomorphic encryption
FPGA

ABSTRACT

The number theoretic transform (NTT) is the most widely used technique for accelerating polynomial multiplication in modern cryptographic systems, including homomorphic encryption. In standard NTT computations, the forward NTT (FNTT) and inverse NTT (INTT) operations are realized using Cooley-Tukey (CT) and Gentleman-Sande (GS) butterfly structures. Additionally, the INTT requires an inverse-scaling operation after the transform. These FNTT, INTT, and inverse-scaling operations exhibit different arithmetic structures and dataflow characteristics, resulting in an inherent structural heterogeneity. However, most existing configurable NTT architectures map these heterogeneous operations onto a single shared datapath within a unified butterfly unit. This leads to complex control logic and inefficient hardware resource utilization. In this paper, we optimize arithmetic resource allocation in configurable NTT hardware to address the structural heterogeneity of FNTT, INTT, and inverse-scaling operations. To realize this approach, we have proposed a configurable NTT hardware architecture that supports multiple polynomial degrees ranging from 2^{12} to 2^{16} and modulus sizes of up to 64 bits through a structurally aware butterfly datapath. The proposed datapath employs three modular multipliers (one dedicated to each CT butterfly, GS butterfly, and INTT scaling operation) together with lightweight routing multiplexers and arithmetic units. Furthermore, we have presented an area-delay-efficient modular multiplier consisting of a novel integer polynomial multiplier and an optimized Montgomery-based modular reduction architecture. The integer multiplier uses parallel partial-product generation, tree-based accumulation, and shifting techniques, while the Montgomery reduction employs operand digitization and a single instance of the integer multiplier for internal multiplication with the modulus q. Finally, we have proposed benchmarking metrics based on the area-delay product for modular multipliers and the logic-normalized coefficient throughput for the NTT accelerator. The NTT architecture is implemented in Verilog HDL using the Vivado design environment and evaluated up to the post-place-and-route stage on an Artix-7 field-programmable gate array device. Comprehensive performance results and comparisons with state-of-the-art designs confirm that the proposed architecture achieves a significantly lower area with clock cycle overhead, demonstrating its suitability for area-constrained applications.

1. Introduction

Cloud computing offers scalable and cost-efficient computation but raises data-security and privacy concerns. Homomorphic encryption (HE) addresses these concerns by enabling computation directly over encrypted data [1,2]. In HE-based systems, client data are encrypted before outsourcing and processed in encrypted form. Authorized users then decrypt the results. Consequently, the data confidentiality is preserved throughout the process. Beyond cloud computing, HE has been adopted in various privacy-sensitive domains such as healthcare analytics [3], financial services [4], and collaborative machine learning [5].

Although HE is becoming an integral part of secure computation, it remains computationally expensive due to its reliance on large-degree polynomial arithmetic [6]. Among these operations, polynomial

⋆ This research was funded by Umm Al-Qura University, Saudi Arabia under grant number: 26UQU4320199GSSR01.
* Corresponding author.
E-mail addresses: ossonbul@uqu.edu.sa (O.S. Sonbul), mfelahi@uqu.edu.sa (M. Rashid), ayjaafar@uqu.edu.sa (A.Y. Jaffar), mjghamdi@uqu.edu.sa (M.J. Alghamdi), m.masud@ubt.edu.sa (M.I. Masud), m.aman@ubt.edu.sa (M. Aman).

https://doi.org/10.1016/j.rineng.2026.111270
Received 21 February 2026; Received in revised form 11 May 2026; Accepted 26 May 2026
Available online 5 June 2026

multiplication is the dominant contributor to the overall computational cost [1,7]. The number theoretic transform (NTT) is frequently used for accelerating polynomial multiplications and consequently serves as a critical block in HE systems. Beyond HE, NTT hardware is also essential in lattice-based PQC schemes [8–12], zero-knowledge proof protocols [13], and privacy-preserving machine learning [14].

The NTT computations involve two operations to implement: forward NTT (FNTT) and inverse NTT (INTT). The FNTT can be efficiently implemented using a Cooley-Tukey (CT) butterfly, while the Gentleman-Sande (GS) butterfly efficiently computes the INTT. In addition, the NTT implementation is primarily controlled by two parameters: the polynomial ring dimension n and the coefficient modulus q. The bit length of the modulus is given by $K = \lceil \log_2 q \rceil$, which directly influences arithmetic complexity. These parameters vary significantly across cryptographic schemes and software libraries. For example, the CRYSTALS-Kyber [15] key-encapsulation mechanism adopts $n = 256$ with a 13-bit modulus q. Similarly, Microsoft SEAL [16] supports ring sizes ranging from $n = 1024$ to 32768 with modulus sizes up to 60 bits [16]. Such diversity in parameter selection necessitates the development of configurable NTT hardware architectures [17].

1.1. Related work in NTT accelerators across various cryptographic domains

Extensive research has focused on designing NTT accelerators for field-programmable gate arrays (FPGAs) and application-specific integrated circuits (ASICs), targeting diverse cryptographic applications, including PQC, HE, and ZKPs. PQC-related accelerators are investigated in [8–10,18]. Similarly, HE-specific NTT architectures have been presented in [1,2,14,19–24], while NTT designs tailored for zero-knowledge proof systems are described in [13,25]. Flexible, scalable, and configurable accelerators are implemented in [7,26–31].

1.1.1. NTT acceleration in lattice-based PQC

Among PQC algorithms, lattice-based schemes, including CRYSTALS-Kyber [15], CRYSTALS-Dilithium [32], and FALCON [33], rely heavily on efficient implementations of FNTT and INTT. Using CRYSTALS-Kyber as a case study, a comparative analysis of iterative and memory-based NTT implementations has been presented in [8]. Recently, in [9], compute-in-memory approaches have been introduced to improve throughput and reduce data movement overhead. In [10], instruction-level acceleration of NTT-related arithmetic has also been explored through custom instruction set extensions on RISC-V processors for the CRYSTALS-Kyber and CRYSTALS-Dilithium algorithms. An FNTT/INTT crypto processor for the FALCON signature scheme has been reported in [18].

1.1.2. NTT acceleration in HE

NTT acceleration has been extensively investigated for HE algorithms, where large polynomial degrees and repeated transform operations dominate execution time. An early dedicated accelerator demonstrated the feasibility of custom NTT hardware for encrypted computation in [19]. The architecture in [14] improved area efficiency and memory utilization through optimized butterfly designs and scheduling strategies, while configurable memory-based designs supporting multiple HE parameter sets were introduced in [20,21]. The most advanced studies are described in [1] and [2]. The work in [1] has emphasized the importance of NTT acceleration for cloud-based privacy-preserving computation. The NTT engine, described in [2], has used system-level GPU-based frameworks to optimize NTT execution and memory management.

The most recent HE-based NTT accelerators are presented in [22–24]. In [22], a hardware-efficient NTT/INTT architecture for fully homomorphic encryption (FHE) is proposed, featuring a novel twiddle factor generator for Radix-2 multi-path delay commutator designs and digital signal processor (DSP)-efficient multipliers via non-standard tiling, reducing DSP utilization without performance loss. In [23], a reconfigurable NTT/INTT accelerator is proposed, supporting flexible polynomial sizes, modulus widths, and processing unit counts. It employs a bit-reversal-free memory access scheme to eliminate read-write conflicts and a shift-optimized Montgomery reduction to reduce multiplier and DSP usage across multiple parameter configurations. In [24], a generic tool for generating Fast Fourier Transform (FFT)/NTT architectures is presented, combining on-the-fly twiddle factor generation with stall-free memory access. Two design strategies are offered: memory-optimized, minimizing Read-Only Memory (ROM) twiddle factor storage, and routing-optimized, enabling higher FPGA clock frequencies. These strategies enable effective customization across applications ranging from low-end PQC to high-end FHE accelerators.

1.1.3. NTT acceleration in ZKP systems

NTT acceleration has received increasing attention in ZKPs systems, where polynomial arithmetic computations form major prover-side bottlenecks. For such bottlenecks, a scalable NTT architecture employing multi-dimensional decomposition techniques has been proposed in [25] to enhance parallelism and scalability. More recently, a multifunctional accelerator for ZKP protocols, such as the Multifunction Tree Unit (MTU), has been proposed in [13] to support multiple cryptographic primitives, including NTT-related operations.

1.1.4. Flexible and reusable NTT architectures

Beyond the previously discussed application-specific NTT designs, flexible and reusable architectures supporting multiple cryptographic domains have been widely studied. A hybrid compile-time and run-time configurable design is proposed in [7] to balance flexibility and efficiency, while a unified FNTT/INTT architecture aimed at improving hardware reuse is presented in [26]. The parameterized and scalable NTT accelerators for both HE and PQC applications are reported in [27–29]. Design-space exploration and HLS-based NTT implementations have been further investigated in [30,31] to analyze architectural trade-offs.

1.2. Limitations of existing NTT architectures

Section 1.1 shows that many FNTT and INTT hardware architectures have been proposed for HE, PQC, and ZKP systems. These include both scheme-specific accelerators and unified, configurable designs that support multiple parameter sets. Most prior work focuses on increasing throughput by using arithmetic parallelism, multiple butterfly units, and heavy resource reuse. These methods improve performance, but they also introduce significant hardware costs. As a result, even with extensive research on fast and scalable NTT architectures, key challenges remain for area-constrained platforms. The main limitations are summarized below:

- **Overuse of arithmetic parallelism:** Many prior designs [2,13,25–27,34,35] use multiple butterfly units or processing elements to increase throughput. These choices raise hardware cost and make the designs unsuitable for resource-constrained systems.
- **Arithmetic reuse with high control overhead:** Architectures such as [8,9,12,14,20,21,28–30] reuse modular multipliers across FNTT, INTT, and inverse-scaling operations. This reduces arithmetic units but introduces complex multiplexing and control logic, increasing routing and logic overhead.
- **Uniform arithmetic for heterogeneous operations:** Unified NTT architectures [8,12,27,28] rely on a single datapath for CT butterflies, GS butterflies, and inverse-scaling. These operations have different dataflow patterns, making efficient mapping onto one datapath difficult.
- **Limited suitability for area-constrained platforms:** The combined overhead of control logic, interconnect complexity, and memory interfacing makes existing configurable NTT accelerators [7,20,21,28–

30] inefficient in settings where minimizing hardware cost is more important than maximizing throughput.

1.3. Our observations and design motivation

We first outline key observations from existing NTT accelerators, followed by the design motivation and objectives of our work.

- **Our observations:** Based on the NTT architectures and their limitations in Sections 1.1 and 1.2, we have observed that the area efficiency depends not only on the number of arithmetic units but also on how they are organized and reused within the datapath. Furthermore, FNTT and INTT computations exhibit inherent structural heterogeneity. The CT butterfly multiplies before adding/subtracting, while the GS butterfly does the opposite, and the INTT adds a final scaling step. These differences lead to different dataflow and timing patterns. Yet many unified designs force these heterogeneous operations onto a single shared datapath, introducing routing, multiplexing, and control overhead that undermines area efficiency.
- **Design motivation of this work:** Motivated by the insights presented above, we optimize arithmetic resource allocation by assigning multipliers to structurally distinct computation paths for the CT butterfly, GS butterfly, and inverse scaling. This strategy improves datapath locality, reduces routing and multiplexing, and simplifies control logic, yielding a lower overall area despite using multiple multipliers in the NTT datapath. It is important to note that this work explicitly occupies the area-optimized end of the NTT hardware design space. Unlike throughput-oriented designs that employ multiple butterfly units, deep pipelining, and aggressive parallelism, the proposed architecture prioritizes minimizing hardware footprint, making it suitable for platforms where FPGA resources or chip area are the critical constraint(s).
- **Objective of this work:** The motivation outlined above can be achieved only if an area-delay-efficient modular multiplier is developed to support the NTT datapath. Therefore, the objective of this work is to design and implement a compact, high-performance modular multiplier that can be used as a dedicated multiplier for each CT, GS, and inverse-scaling operation while reducing the overall area of the configurable NTT accelerator.

1.4. Contributions

The contributions are summarized as follows:

- **Optimizing arithmetic resource allocations:** We have reconsidered the arithmetic resource allocation in configurable NTT hardware by assigning multipliers to structurally distinct computation paths rather than aggressively minimizing their number. This optimization highlights structural heterogeneity in NTT computations in Section 2.5.
- **Compile-time configurable design:** We have proposed a compile-time configurable NTT architecture that supports multiple polynomial degrees (ranging from 2^{12} to 2^{16}) and large modulus sizes (up to 64-bits) through a structurally aware unified butterfly datapath. Rather than aggressively minimizing the number of multipliers through a shared datapath (which introduces complex multiplexing and control logic overhead, as observed in prior sequential designs [8,20,21]), the butterfly datapath assigns three dedicated modular multipliers, one to each of the CT butterfly, GS butterfly, and INTT scaling paths. This structural assignment eliminates reconfiguration overhead between heterogeneous arithmetic phases, enabling a clean iterative implementation that achieves lower area than designs using equivalent or fewer multipliers with shared datapaths. The overview of the NTT architecture and our structurally aware unified butterfly datapath are detailed in Sections 3.1 and 3.2.
- **Area-delay-efficient modular multiplier:** In Section 3.2.1, we proposed an area-delay-efficient modular multiplier that comprises an integer polynomial multiplier as well as an efficient Montgomery-based modular reduction architecture.
- **Integer polynomial multiplication design:** A hardware optimized digit-decomposed integer multiplication architecture is proposed for FPGA implementation, employing parallel partial-product generation, tree-based accumulation, and digit-level shifting. While digit decomposition is a classical concept in computer arithmetic, the proposed implementation is specifically engineered for FPGA datapath locality, achieving significantly lower logic and routing delays compared to conventional schoolbook, direct HDL (*), and recursive Karatsuba implementations on the same target device. The resulting area-delay efficiency is demonstrated quantitatively in Section 4.1.1.
- **Montgomery-based modular reduction design:** We have proposed a Montgomery-based modular reduction architecture through an operand digitization approach together with one instance of our proposed integer multiplier in its datapath for the internal multiplication with modulus q.
- **Performance benchmarking on FPGA.** We comprehensively compared the proposed modular multiplier and the NTT architecture after the post-place-and-route stage on an FPGA. Moreover, we defined a benchmarking metric for evaluating the modular multipliers in terms of the area-delay product and the NTT architecture in terms of the logic-normalized coefficient throughput.

The comprehensive performance results, benchmarking, and comparisons in Sections 4.1–4.3 show that, despite using multiple arithmetic operators for heterogeneous NTT operations, our architecture achieves significantly lower area than state-of-the-art configurable NTT accelerators for polynomial degrees from 2^{12} to 2^{16}. This area efficiency makes our design well-suited for area-constrained homomorphic encryption applications.

The remainder of this paper is structured as follows. The mathematical background on NTT is discussed in Section 2. The proposed NTT accelerator architecture is presented in Section 3. Implementation results, benchmarking, and comparisons with state-of-the-art NTT accelerators are provided in Section 4. Finally, Section 5 concludes the paper.

2. Preliminaries

This section investigates the mathematical foundations of the NTT and examines the structural characteristics of its core computational components. The FNTT and INTT structures are first presented in Section 2.1. The arithmetic structures of the CT and GS butterfly configurations are then described in Sections 2.2 and 2.3, respectively. The inverse scaling operation required at the end of the INTT computation is discussed in Section 2.4. Finally, Section 2.5 analyzes these operations collectively to highlight the inherent structural heterogeneity of NTT computation, which motivates the architectural design choices presented in the subsequent section.

2.1. FNTT and INTT

The FNTT maps an n-degree polynomial $f(x) = \sum_{i=0}^{n-1} f_i x^i$ into the NTT domain as

$$\hat{f} = \mathrm{NTT}(f), \tag{1}$$

where each transformed coefficient is given by

$$\hat{f}_i = \sum_{j=0}^{n-1} f_j \zeta^{(2i+1)j}, \tag{2}$$

and ζ denotes a $2n$th primitive root of unity modulo the prime modulus q. Similarly, the INTT reconstructs the polynomial from the NTT transform domain to the normal domain as

$$f_i = n^{-1} \sum_{j=0}^{n-1} \hat{f}_j \zeta^{-i(2j+1)}, \tag{3}$$

where n^{-1} represents the multiplicative inverse of n modulo q. In other words, it represents the inverse scaling operation required at the end of the INTT computation.

2.2. Cooley-Tukey butterfly structure

The FNTT is commonly implemented using the Cooley-Tukey butterfly configuration. For a butterfly operating on inputs u and t, the CT butterfly computes

$$y_0 = (u + t \cdot \zeta) \bmod q, \quad y_1 = (u - t \cdot \zeta) \bmod q, \tag{4}$$

where ζ denotes the twiddle factor corresponding to the current stage. In this formulation, the multiplication by the twiddle factor precedes the addition and subtraction operations. As a result, the critical arithmetic path of the CT butterfly is dominated by a modular multiplication followed by modular addition and subtraction.

2.3. Gentleman-Sande butterfly structure

The INTT is typically implemented using the Gentleman-Sande decomposition. Given two input coefficients u and t, the GS butterfly computes

$$y_0 = (u + t) \bmod q, \quad y_1 = (u - t) \cdot \zeta \bmod q. \tag{5}$$

Unlike the CT butterfly, the GS structure performs addition and subtraction first, followed by twiddle-factor (i.e., ζ) multiplication applied to only one output branch. Consequently, the multiplier appears at a different datapath location than the CT butterfly, and the timing and data dependencies of arithmetic operations are fundamentally different.

2.4. Inverse scaling operation

In addition to GS butterfly computations, the inverse NTT requires a global scaling operation after the completion of all butterfly stages. Each coefficient must be multiplied by n^{-1} modulo q, as shown in Eq. (3). This operation is independent of the butterfly structure and is applied uniformly to all coefficients. The scaling step constitutes a distinct arithmetic phase with different control flows and memory access patterns compared to both CT and GS butterflies.

2.5. Structural heterogeneity in NTT

From the mathematical formulations of Eqs. (4) and (5), and the inverse scaling operation, it is evident that FNTT and INTT computations exhibit inherently heterogeneous arithmetic structures. The CT butterfly performs twiddle-factor multiplication prior to addition and subtraction, whereas the GS butterfly reverses this order by applying multiplication after the arithmetic operations. In addition, the INTT requires a separate post-scaling multiplication beyond those involved in the CT and GS butterfly structures. These differences result in fundamentally distinct computation patterns across the three operations.

Note that from a hardware design perspective, mapping these heterogeneous arithmetic structures onto a single shared datapath can lead to complex control logic, extensive multiplexing, and inefficient routing. This limitation is evident in several prior NTT architectures reported in the literature [8,9,12,14,20,21,28–30].

Our insight is that enforcing structural diversity onto a unified arithmetic datapath undermines area efficiency and negates the potential benefits of resource sharing. Consequently, we conclude that heterogeneous NTT operations are better served by dedicated arithmetic resources rather than by a highly multiplexed shared datapath. Guided by this conclusion, our design employs three distinct modular multipliers tailored for the CT butterfly, GS butterfly, and inverse scaling operations. Based on this reorganization, the proposed NTT architecture is presented in Section 3.

3. Proposed compile-time configurable NTT architecture

Using the mathematical formulations and structural analysis from Section 2, this section describes the proposed compile-time configurable FNTT/INTT hardware architecture. The overview of the architecture is introduced in Section 3.1, followed by a detailed description of the configurable and unified butterfly datapath in Section 3.2, and the supporting memory and control subsystems in Section 3.3. Together, these architectural components explicitly reflect the structural heterogeneity of the CT butterfly, GS butterfly, and inverse scaling operations identified earlier in Section 2.5.

3.1. Architecture overview

Fig. 1 shows the top-level architecture of the proposed compile-time configurable FNTT/INTT accelerator. As our design is intended for area-constrained HE platforms, it adopts an iterative strategy to implement Eqs. (2) and (3) for FNTT and INTT computations, respectively. Moreover, our architecture reuses a single unified butterfly datapath across $\log_2(n)$ stages, with a higher cycle count overhead compared to fully parallel architectures. This iterative strategy represents an intentional area-throughput trade-off: it minimizes hardware footprint at the cost of increased clock-cycle count, and is specifically motivated by the requirements of area-constrained HE platforms where silicon or FPGA resources are limited. The latency implications of this choice are analyzed and reported in Section 4.2 and Table 3. Compile-time configurability is achieved through a Verilog header file that defines four synthesis-time parameters: (i) the polynomial ring size n, (ii) the coefficient bit-width $K = \lceil \log_2 q \rceil$, (iii) the prime modulus q, and (iv) the modular inverse $n^{-1} \bmod q$ used for inverse scaling. Fixing these parameters at synthesis time eliminates run-time reconfiguration overhead and enables efficient use of memory depths, address counters, and datapath widths for a targeted parameter set.

The accelerator operates in two modes controlled by an external signal: mode1. When mode1 = 0, the datapath performs the FNTT using the Cooley-Tukey butterfly structure described in Section 2.2. When mode1 = 1, the datapath performs the INTT using the Gentleman-Sande butterfly structure described in Section 2.3, followed by the inverse scaling operation discussed in Section 2.4. This explicit mode separation enables a unified top-level architecture while allowing the internal datapath to accommodate the distinct arithmetic ordering and operand timing of CT, GS, and inverse scaling operations.

At the system level, the accelerator comprises four main components: (i) a unified butterfly datapath, (ii) a memory subsystem (MemBlock), (iii) an address generation unit (AddressGen), and (iv) a finite-state machine (FSM)-based controller. The unified butterfly datapath implements both CT and GS arithmetic, as well as the inverse scaling operation, through dedicated internal routing and arithmetic placement; its structure and operation are detailed in Section 3.2. The MemBlock stores polynomial coefficients and twiddle factors. It consists of two dual-port coefficient memories used in a ping-pong manner to support simultaneous read and write operations during iterative processing, and one single-port memory for twiddle-factor storage. The AddressGen unit produces stage-dependent read/write addresses for coefficient memories and synchronizes twiddle-factor addressing with butterfly execution. The FSM controller sequences the iterative schedule over all stages, configures internal routing, and orchestrates data movement across the memory subsystem. The MemBlock and AddressGen and FSM units are further detailed in Section 3.3.

To support large-modulus arithmetic required by practical HE parameter sets, the modulus datapath is implemented with a fixed 64-bit width. When smaller moduli are used, like $q < 2^{64}$, the modulus value is zero-extended to 64 bits. This design choice keeps the arithmetic datapath and DSP utilization constant across supported parameter sets and avoids repeated synthesis-time restructuring of the modular multiplication datapath. The modular multiplication relies on an integer mul-

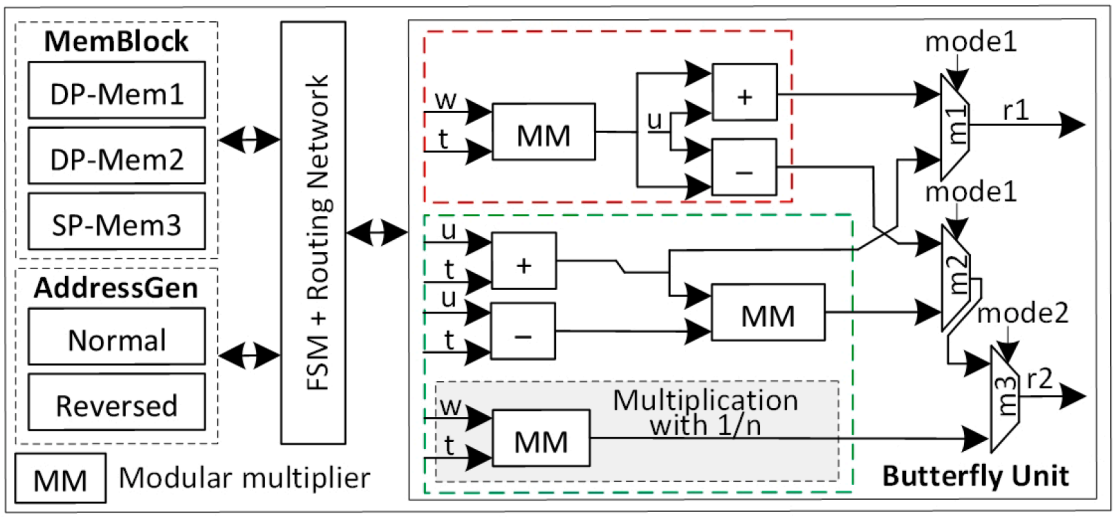

Fig. 1. Top-level design of our proposed compile-time configurable NTT accelerator.

tiplication architecture integrated with a Montgomery-based modular reduction, which is presented in Section 3.2.1. Overall, the architecture emphasizes datapath locality and simplified control, enabling low hardware cost while retaining compile-time configurability across multiple (n, q) design parameters.

3.2. Structurally aware unified butterfly datapath

The unified butterfly datapath is the computational core of the proposed NTT architecture. It supports both FNTT and INTT operations within a single architectural framework using the heterogeneous arithmetic structures of the CT and GS butterfly formulations (introduced in Sections 2.2 and 2.3).

As previously reported in Section 2, the CT butterfly for FNTT computation applies twiddle-factor multiplication before addition and subtraction, whereas the GS butterfly for INTT reverses this arithmetic order by performing multiplication after the arithmetic operations. Moreover, Section 2.4 reports that at the end of the INTT, an additional coefficient scaling operation is required. These operations exhibit different operand ordering, timing constraints, and data dependencies, which complicate their efficient realization using a single shared arithmetic datapath.

To accommodate this structural heterogeneity, the proposed architecture adopts a structurally aware butterfly architecture. Rather than aggressively reusing a single modular multiplier across all computation phases, three dedicated modular multiplication paths are instantiated within the unified butterfly unit datapath: one assigned to the CT butterfly for FNTT computation, one assigned to the GS butterfly for INTT computation, and one reserved for the inverse-scaling operation. This allocation aligns each multiplier with a fixed arithmetic role, enabling localized dataflow and eliminating the need for frequent reconfiguration of arithmetic units.

As illustrated in Fig. 1, the red-outlined portion of the datapath corresponds to the CT butterfly computation, while the green-outlined portion implements the GS butterfly computation. The outputs of these two computation paths are selected using lightweight 2×1 routing multiplexers, and the required addition and subtraction operations are implemented using standard HDL operators. By assigning fixed arithmetic roles to each modular multiplier, the proposed datapath minimizes control complexity and routing overhead compared to architectures that rely on aggressive arithmetic reuse across structurally heterogeneous NTT operations, thereby achieving improved area efficiency despite employing three modular multipliers. The internal architecture of the modular multiplier, including the proposed integer multiplication architecture and the Montgomery-based modular reduction scheme, is described in the next Section 3.2.1.

3.2.1. Proposed area–delay efficient modular multiplier

Modular multiplication is the dominant arithmetic operation within the unified butterfly datapath. As discussed in Section 3.2, twiddle-factor multiplication is required in both CT and GS butterfly computations, while an additional multiplication by n^{-1} is performed during the inverse scaling stage. Consequently, the efficiency of the overall NTT accelerator is strongly influenced by the design of the modular multiplier.

To support large-modulus arithmetic required by practical HE parameter sets, the proposed butterfly integrates a modular multiplier capable of operating on operands of up to 64 bits. The modular multiplier, shown in Fig. 2, is composed of two main components: (i) a 64-bit integer multiplication architecture and (ii) a Montgomery-based modular reduction stage. These components are tightly integrated to achieve low logic and routing delay while maintaining predictable timing behavior suitable for iterative NTT implementation. In other words, the integer multiplication architecture focuses on reducing critical-path delay by exploiting digit-level parallelism, whereas the modular reduction stage employs Montgomery arithmetic to avoid expensive division operations. The internal structure of these two components is described in the following paragraphs.

Integer multiplication (IM) architecture. As illustrated in Fig. 2, the integer multiplication architecture employs a digit-decomposition strategy to reduce critical-path delay and improve parallelism. As the 64-bit input operands a and b are first partitioned into four 16-bit digits each. The resulting 16×16-bit digit-level multiplications are performed fully in parallel, as indicated by the orange blocks. These parallel multipliers generate a set of partial products corresponding to all digit combinations. The generated partial products are stored in a two-dimensional array of registers, denoted as the partial-product (PP) array and highlighted by the green blocks in Fig. 2. Each partial product is then shifted according to its digit position and stored in a second register array, referred to as the partial-shift (PS) array, shown in blue. This shifting aligns the partial products to their appropriate bit significance before accumulation. Following alignment, the shifted partial products are accumulated in parallel using pairwise addition, as represented by the gray blocks in Fig. 2. The accumulation process produces the final 128-bit intermediate result, denoted as FP, which represents the full-precision product of the two 64-bit operands.

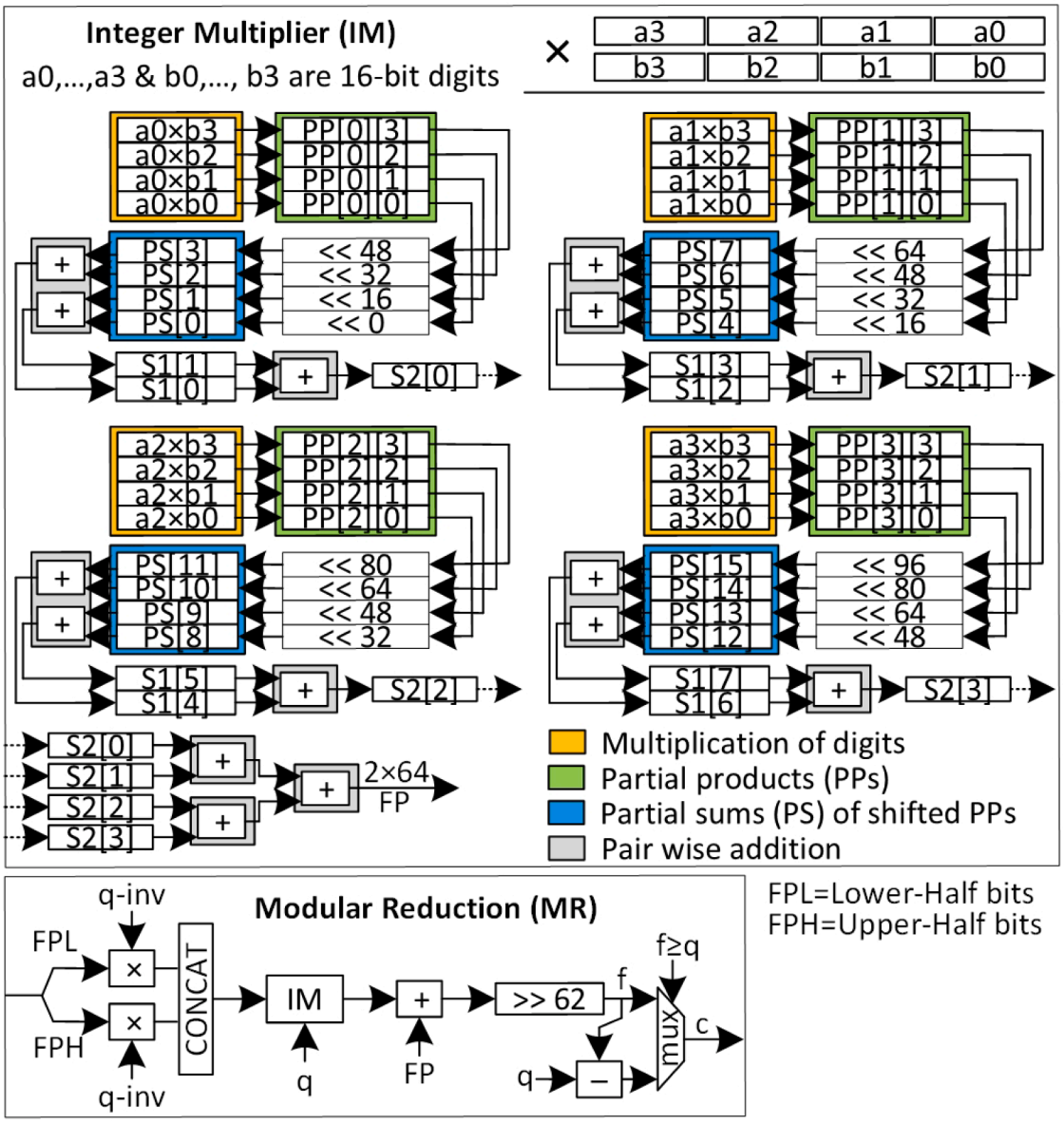

Fig. 2. Architecture of the proposed 64 × 64-bit modular multiplier integrated within the unified butterfly datapath. The design combines a digit-decomposed integer multiplication structure with parallel partial-product generation, shifting, and accumulation, followed by a Montgomery-based modular reduction stage to support efficient large-modulus arithmetic.

By enabling concurrent digit multiplication, shifting, and accumulation, the proposed integer multiplier significantly shortens the critical computation path. This parallel structure results in lower logic and routing delays compared to conventional schoolbook multipliers, direct HDL-based multiplication using the (*) operator, and recursive Karatsuba multiplication architectures. A detailed quantitative comparison is provided in Section 4.1.1.

It is worth noting that the digit-decomposition strategy for integer multiplication in this work is related to high-radix multiplication principles well established in computer arithmetic literature. The distinction of the proposed implementation lies in its FPGA-specific architectural realization: the 4 × 4 digit multiplication grid, pipelined partial-product registers, and 4-stage binary tree accumulator are co-designed to minimize routing congestion and critical-path delay on LUT-based FPGA fabrics. Furthermore, as shown in Fig. 2, the integer multiplier is instantiated twice within the full modular multiplier: (i) once for the primary operand multiplication and (ii) once internally within the Montgomery reduction stage for multiplication by the modulus q, making the overall modular multiplier performance area-delay efficient.

Montgomery-based modular reduction:. Following integer multiplication, the resulting 128-bit intermediate product must be reduced modulo the prime modulus q. As illustrated in Fig. 2, this operation is performed using a Montgomery-based modular reduction scheme, which enables efficient reduction without explicit division. The 128-bit intermediate result FP is first partitioned into two 64-bit segments: the lower word FPL and the upper word FPH. These segments are processed in parallel through multiplication with a precomputed q-inv constant derived from the modulus q, as shown in Fig. 2. The resulting values are then concatenated using the CONCAT block to form an intermediate Montgomery product. This intermediate value is subsequently multiplied by the modulus q using the proposed integer multiplication architecture. The output of this multiplication is added to the original product FP, and the result is right-shifted to generate an intermediate reduced value, denoted as f. Finally, a conditional subtraction is performed: if f is greater than or equal to q, the final modular result c is computed as f − q; otherwise, c is assigned directly as f.

By relying solely on multiplication, addition, and conditional subtraction operations, the Montgomery-based reduction avoids costly divi-

sion and enables efficient large-modulus arithmetic within the butterfly datapath.

3.3. Memory block and control logic

Efficient data storage and access are essential for iterative NTT computation, particularly when a single butterfly unit is reused across all stages. To support this execution model, the proposed architecture incorporates a MemBlock together with an FSM-based controller and an AddressGen unit, as shown in Fig. 1.

The MemBlock consists of three block RAMs as memory blocks (DP-MEM1, DP-MEM2 and SP-MEM3). Two dual-port memories (DP-MEM1 and DP-MEM2) are employed to store initial, intermediate, and final polynomial coefficients and operate in a ping-pong fashion, enabling simultaneous read and write operations during each butterfly computation. This allows continuous data streaming through the butterfly datapath while avoiding read-write conflicts. In addition, a single-port memory SP-MEM3 is used to store the corresponding twiddle factors required for both FNTT and INTT computations.

An address generation unit (i.e., AddressGen in Fig. 1) is responsible for producing the appropriate read and write addresses for each butterfly stage. During FNTT execution, addresses are generated in normal order, whereas inverse NTT computation employs bit-reversed addressing to satisfy the data access patterns of the GS butterfly formulation. The AddressGen unit operates iteratively across $\log_2 n$ stages and is parameterized in our architecture according to the configured ring size.

The overall operation of the NTT accelerator is coordinated by an FSM-based controller. It manages butterfly scheduling, memory access sequencing, stage transitions, and mode selection between FNTT and INTT execution. By centralizing control in an FSM and reusing a single butterfly datapath, the proposed design achieves predictable timing behavior and low hardware overhead while supporting a wide range of NTT parameters.

4. Implementation results, performance benchmarking and comparison with state-of-the-art

All architectures were implemented in synthesizable Verilog HDL and evaluated using the Xilinx Vivado IDE 2023.2 toolflow. The designs were synthesized, placed, and routed up to the post-place-and-route (PPnR) level on a Virtex-7 (VX690T) FPGA device to ensure accurate estimation of hardware resource utilization and timing characteristics. Moreover, this section first presents the implementation results of the proposed configurable NTT accelerator, including area consumption, timing performance, and resource breakdown in Section 4.1. Subsequently, performance benchmarking is discussed in Section 4.2, where clock-cycle counts and latency characteristics of the FNTT and INTT operations are analyzed across different ring sizes. Finally, a comprehensive comparison with state-of-the-art NTT architectures is provided in Section 4.3, highlighting the relative advantages of the proposed design in terms of area efficiency and overall hardware cost.

4.1. Implementation results

The implementation results of the proposed modular multiplication architecture and the complete NTT accelerator are evaluated in Sections 4.1.1 and 4.1.2, respectively. Hardware resource utilization is reported in terms of slices, look-up tables (LUTs), DSP blocks, and block RAMs (BRAMs) for the NTT design. For the modular multiplication architecture, timing performance is evaluated based on logic and routing delays, whereas for the NTT accelerator, performance is analyzed in terms of clock cycle count and overall latency corresponding to the FNTT and INTT computations. The latency is calculated using Eq. (6).

$$\text{Latency } (\mu s) = \frac{\text{Clock Cycles}}{\text{Frequency (MHz)}} \tag{6}$$

4.1.1. Results for modular multiplication architectures

The proposed integer multiplication architecture, integrated with a Montgomery-based modular reduction, was implemented as an independent hardware block and evaluated at the post-place-and-route (PPnR) level to accurately characterize its area consumption and timing behavior.To assess its effectiveness, the proposed optimized design is compared against baseline architectures developed in this work, including partial-product-based schoolbook multiplication, direct multiplication using the HDL (*) operator, and recursive Karatsuba multiplication with splitting up to level four. Table 1 summarizes the area utilization and datapath delay of these 64×64-bit non-pipelined multipliers under identical implementation conditions. For completeness and fairness, the evaluation is conducted under two scenarios: (i) with DSP blocks enabled and (ii) with DSP blocks disabled, thereby isolating the impact of logic and routing characteristics on overall performance. In addition, the area×delay product is computed in LUT·ns for efficiency.

Comparing proposed optimized with baseline architectures developed in this work:. When DSP blocks are enabled, the proposed multiplier achieves a substantially lower datapath delay compared to all baseline implementations. The proposed design exhibits balanced logic and routing delays of 3.073 ns and 3.040 ns, respectively, resulting in a total delay of approximately 6.1 ns. In contrast, the total datapath delays of the schoolbook, direct HDL-based, and recursive Karatsuba multipliers exceed 38 ns, 39 ns, and 51 ns, respectively, corresponding to a delay reduction of approximately 84%-88% relative to conventional approaches. Furthermore, while routing delay contributes roughly 50% of the critical path in the proposed architecture, it dominates the datapath delay in existing designs, accounting for up to 60% in schoolbook multiplication and as high as 66% in recursive Karatsuba multiplication. This behavior indicates that conventional partial-product accumulation structures suffer from long interconnect paths and limited datapath locality when mapped onto FPGA fabrics. By contrast, the proposed digit-based multiplication architecture significantly improves locality and mitigates routing congestion, resulting in a more balanced timing profile and improved scalability for large-operand arithmetic. In addition, this behavior is consistent with the architectural objective of the proposed integer multiplier: rather than introducing new mathematical primitives, the design contributes an FPGA-optimized realization of digit-level parallelism that achieves measurably superior area-delay efficiency over all evaluated baselines. This reflects our engineering effort that efficiently maps a structured multiplication strategy onto LUT-and-DSP FPGA fabrics for large-modulus cryptographic arithmetic.

A similar trend is observed when DSP blocks are disabled. In this configuration, the proposed multiplier achieves a total datapath delay of approximately 6.45 ns, compared to 48.6 ns, 50.8 ns, and 53.0 ns for the schoolbook, direct HDL-based, and recursive Karatsuba multipliers, respectively. This corresponds to a delay reduction of approximately 86%-88% relative to conventional approaches. Moreover, routing delay dominates the critical path in baseline designs, contributing more than 75% of the total delay in all cases, whereas the proposed architecture limits the routing contribution to approximately 59%. Despite the increased logic utilization associated with DSP-free implementations, the segmented digit-based multiplication strategy significantly reduces interconnect complexity, resulting in improved datapath locality and a more balanced timing profile. In terms of LUT·ns, the proposed optimized design, including and excluding the DSP block configurations, is more efficient than all baseline architectures developed in this work.

Comparing proposed optimized with optimized 64×64 bit designs of [36]:. Three optimized 64-bit integer multipliers are described in [36] on the Xilinx FPGA. It is important to note that these designs report standalone

Table 1
Area and datapath delay evaluation of 64 × 64 non-pipelined integer multipliers using Montgomery-based modular reduction, implemented on a VX690T FPGA. Recursive Karatsuba is implemented splitting up to level four. Designs marked with † are for integer multiplication only (without Montgomery reduction) and report synthesis-only results on Kintex-7 from [36]. The **bold** text highlights the superior performance of the proposed multiplier in terms of logic and routing delays and LUT×Delay product.

Evaluation Mode	Designs	Slices / LUTs / DSPs	Logic / Routing Delays (ns)	T. Delay (ns)	LUT×T. Delay (LUT·ns)
With DSP	**Baseline Architectures Developed in This Work**				
	Schoolbook	1388 / 5077 / 26	15.475 (40%) / 23.463 (60%)	38.938	197,726
	Direct (with * in HDL)	207 / 726 / 42	18.603 (48%) / 20.252 (52%)	38.855	28,208
	Recursive Karatsuba	1290 / 4424 / 26	17.130 (34%) / 33.983 (66%)	51.113	226,124
	Optimized Architectures from State-of-the-Art				
	Xilinx IP [36][a]	– / 209 / 16	– / –	11.705	2,446
	DSP(16) + CSA + CPA [36][a]	– / 751 / 16	– / –	8.448	6,345
	DSP(12) + CSA + CPA [36][a]	– / 624 / 12	– / –	8.335	5,201
	Proposed Optimized	335 / 1059 / 32	**3.073** (50%) / **3.040** (50%)	**6.113**	**6,474**
Without DSP	**Baseline Architectures Developed in This Work**				
	Schoolbook	3796 / 12,379 / 0	11.249 (23%) / 37.338 (77%)	48.587	601,400
	Direct (with *)	4159 / 12,918 / 0	11.838 (23%) / 38.933 (77%)	50.771	656,063
	Recursive Karatsuba	3663 / 11,727 / 0	13.100 (25%) / 39.915 (75%)	53.015	621,672
	Proposed Optimized	1610 / 5170 / 0	**2.649** (41%) / **3.801** (59%)	**6.450**	**33,347**

[a] Integer multiplication only (no Montgomery reduction); synthesis-only results on Kintex-7 (xc7k70t).

integer multiplication results only, without Montgomery-based modular reduction, and are evaluated at synthesis level on a Kintex-7 device, whereas all other designs in Table 1 include full Montgomery reduction and are evaluated at the post-place-and-route stage on a Virtex-7. Despite this fundamental difference in functional scope, the proposed multiplier achieves a total delay of 6.113 ns, which is 46.8% lower than the 11.705 ns reported for the Xilinx IP [36], confirming that the proposed design is faster even against a vendor-optimized baseline that performs only integer multiplication without modular reduction. When compared against the two CSA-based designs, the proposed multiplier achieves a comparable area×delay product of 6,474 LUT·ns against 5,201–6,345 LUT·ns, despite the additional overhead of a complete Montgomery-based modular reduction stage that is not considered in all three designs of [36]. This demonstrates that the proposed architecture delivers state-of-the-art 64-bit integer multiplication efficiency while simultaneously integrating modular reduction, confirming its suitability as an area-delay-efficient arithmetic building block for the configurable NTT accelerator.

Apart from the designs of [36], it is worth noting that TTech-LIB [37,38] provides an open-source collection of optimized integer polynomial multipliers for FPGA, targeting elliptic curve cryptography with operand sizes in powers of two. Although TTech-LIB addresses a different application domain and operand size range from the 64-bit modular arithmetic required in HE-based NTT, it serves as a valuable reference platform for researchers working on large integer polynomial multiplication on FPGAs.

4.1.2. Results for NTT architecture

Table 2 reports the performance and resource utilization of the proposed FNTT/INTT architecture for ring sizes ranging from 2^{12} to 2^{16}. As the ring size increases, the design exhibits predictable and consistent scaling trends in both hardware resources and execution latency.

From an area perspective, slice utilization increases from 1446 slices at 2^{12} to 3809 slices at 2^{16}, corresponding to an overall increase of approximately 2.63×. Similarly, LUTs usage grows from 4285 to 10,783 LUTs, resulting in a 2.52× increase. This moderate growth reflects the impact of larger on-chip memories and control logic required to support higher-degree polynomial operations. In contrast, DSP utilization remains constant at 97 DSP blocks across all configurations, as the architecture employs fixed 64-bit modular arithmetic for the prime modulus q. Even when smaller moduli ($q < 64$ bits) are used, the same DSP resources are retained, with unused higher bits padded with zeros, thereby preserving a uniform datapath structure.

Table 2
Performance results of our FNTT/INTT design on VX690T FPGA @ 100MHz with a prime modulus of $\log_2 q = 64$ bit. The utilized BRAMs size is 36KB.

Design	RingSize (in 2^n, where n = 12 to 16)				
Metrics	2^{12}	2^{13}	2^{14}	2^{15}	2^{16}
Slices	1446	1441	1520	1643	3809
LUTs	4285	4314	4499	4608	10,783
DSPs	97	97	97	97	97
BRAMs	22.5	48	96	192	384
Clock Cycles (CCs) and Latency (in μs) for FNTT					
CCs	49,190	106,537	229,420	491,567	1,048,626
Latency	491	1065	2294	4915	10,486
Clock Cycles (CCs) and Latency (in μs) for INTT					
CCs	77,876	167,992	360,508	770,112	1,638,468
Latency	778	1679	3605	7701	16,384
Average-power (in mW) for FNTT and INTT					
Static	166	167	171	173	175
Dynamic	261	382	558	816	1193
Total	427	549	729	989	1368

BRAMs utilization scales linearly with the ring size, increasing from 22.5 BRAMs at 2^{12} to 384 BRAMs at 2^{16}, representing a 17.1× increase. This behavior directly corresponds to the proportional growth in polynomial storage requirements. For $n = 2^{12}$, a fractional BRAM usage of 22.5 indicates that 22 BRAM blocks are configured as 36 KB memories, while one BRAM is utilized as an 18 KB block.

From a performance standpoint, the number of clock cycles (CCs) required for the FNTT increases from 49,190 cycles at 2^{12} to 1,048,626 cycles at 2^{16}, resulting in a 21.3× increase. A similar trend is observed for the INTT, where the cycle count rises from 77,876 to 1,638,468 cycles, corresponding to an increase of approximately 21.0×. This scaling behavior closely follows the expected $O(N \log N)$ computational complexity of the NTT algorithm.

At an operating frequency of 100 MHz, the corresponding latency for FNTT increases from 491 μs to 10,486 μs as the ring size grows from

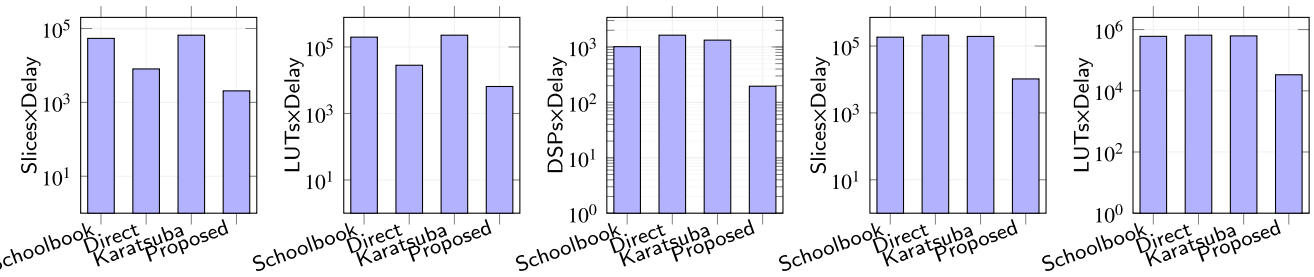

Fig. 3.1 With DSP blocks Fig. 3.2 Without DSP blocks

Fig. 3. Area–delay product benchmarking of 64 × 64 non-pipelined modular multipliers. Fig. 3.1 reports Slices×Delay, LUTs×Delay, and DSPs×Delay when DSP blocks are enabled. Fig. 3.2 reports Slices×Delay and LUTs×Delay for purely logic-based implementations (DSP usage is zero for all designs and therefore omitted). Lower the value of the ADP, higher will be the performance of the architecture.

2^{12} to 2^{16}. Likewise, INTT latency increases from 778 μs to 16,384 μs over the same range. Across all configurations, INTT consistently exhibits higher latency than FNTT due to two additional operations: (i) bit-reversal address generation during the inverse transformation and (ii) the coefficient scaling operation by $\frac{1}{n}$ (or n^{-1}) required after the final stage of INTT computation, as discussed in Section 2.4.

Regarding the average power consumption of the proposed FNTT/INTT architecture on the VX690T FPGA at 100 MHz for polynomial sizes ranging from $n = 2^{12}$ to 2^{16}, the static power remains nearly constant across all configurations, ranging from 166 mW to 175 mW, reflecting the fixed platform overhead independent of the design activity. The dynamic power scales with the polynomial size, increasing from 261 mW at $n = 2^{12}$ to 1193 mW at $n = 2^{16}$, as larger transforms require more active switching activity over extended clock cycles. The total power consumption ranges from 427 mW to 1368 mW, remaining within a practical range for cryptographic accelerators deployed in resource-constrained environments. These results confirm the power efficiency of the proposed architecture.

It is important to note that the proposed design prioritizes area efficiency over throughput by employing a single butterfly unit and sequential processing. As a result, the observed latency can be significantly reduced by instantiating multiple butterfly units and increasing parallelism, similar to the approaches reported in [27,28]. Such optimizations would improve throughput at the expense of increased area and memory bandwidth, highlighting the trade-off space between resource utilization and performance in NTT hardware design.

4.2. Performance benchmarking

To ensure a fair and realistic comparison, the multiplication architectures are benchmarked in Section 4.2.1, while the complete NTT architecture is evaluated separately in Section 4.2.2.

4.2.1. Multiplier performance benchmarking

In Fig. 3, the performance of different modular multiplier architectures is evaluated using a composite benchmarking metric that captures both hardware cost and computational speed. As all multipliers are non-pipelined and generate output within a single clock cycle, overall performance is dominated by the critical-path delay. Accordingly, the area-delay product (ADP), defined in Eq. (7), is adopted as the primary benchmarking metric, with area measured in terms of slices, LUTs, and DSP blocks. This metric provides a unified basis for comparing architectural efficiency by quantifying the trade-off between resource utilization and achievable operating frequency.

$$\text{ADP} = \text{Area} \times \text{Total Delay} \tag{7}$$

Including DSP blocks. The first three panels (from left to right) in Fig. 3 present the ADP benchmarking results for multiplier implementations utilizing DSP blocks. The evaluation includes three metrics: Slices×Delay, LUTs×Delay, and DSPs×Delay. The first panel (on the left) reveals that the proposed multiplier achieves a substantial reduction in slice-based ADP compared to conventional approaches. Specifically, the proposed design reduces Slices×Delay by approximately 26.4× compared to the schoolbook multiplier and by 32.2× relative to the recursive Karatsuba implementation. Even when compared with the direct HDL multiplication using the built-in (*) operator, the proposed architecture achieves an improvement of nearly 3.9×.

A similar trend is observed when LUT-based ADP is evaluated. The proposed design achieves a reduction of approximately 30.5× compared to the schoolbook approach and 34.9× compared to the recursive Karatsuba. This improvement is primarily attributed to the significantly shorter critical path, resulting from balanced datapath partitioning and reduced routing congestion.

The third panel from left to right in Fig. 3 reports DSPs×Delay, highlighting the efficiency of DSP utilization. Although the proposed architecture employs a moderate number of DSPs blocks, its substantially lower delay results in the smallest DSP-based ADP among all evaluated designs, achieving approximately 5.2× lower DSPs×Delay compared to the schoolbook multiplier and 6.8× lower than the Karatsuba design. These results demonstrate that effective datapath optimization can outweigh modest increases in DSP usage when minimizing overall latency.

Without DSP blocks. The last two panels from left to right in Fig. 3 illustrate the benchmarking results for purely logic-based multiplier implementations, where DSP blocks are disabled. The figure shows that the proposed multiplier significantly outperforms conventional logic-based designs, achieving a reduction of approximately 17.8× in Slices×Delay compared to the schoolbook implementation and 18.7× relative to recursive Karatsuba. Similarly, the last panel in Fig. 3 shows that the proposed architecture reduces LUTs×Delay by approximately 18.0× and 18.6× compared to schoolbook and Karatsuba multipliers, respectively.

Remarks. The proposed modular multiplier achieves significantly higher efficiency across both DSP-enabled and logic-only configurations. Based on the LUTs×Delay metric, the proposed design attains improvements of up to 30.5×, 4.4×, and 34.9× over the schoolbook, direct HDL-based, and recursive Karatsuba multipliers, respectively, when DSP blocks are enabled. Without DSP resources, it still achieves improvements of up to 18.0×, 19.7×, and 18.6× over the corresponding designs. These results confirm the effectiveness of the routing-aware and critical-path-optimized datapath, making the proposed multiplier well suited for latency-constrained cryptographic accelerators.

4.2.2. Benchmarking NTT architecture

To evaluate the efficiency of the proposed NTT architecture using a single unified metric, we define the logic-normalized coefficient throughput (η_{NTT}) using Eq. (8), where n denotes the ring size, $L_{\mu s}$ is the measured latency in microseconds, and A_{logic} represents the logic area measured in terms of LUTs. The factor 10^6 converts the latency

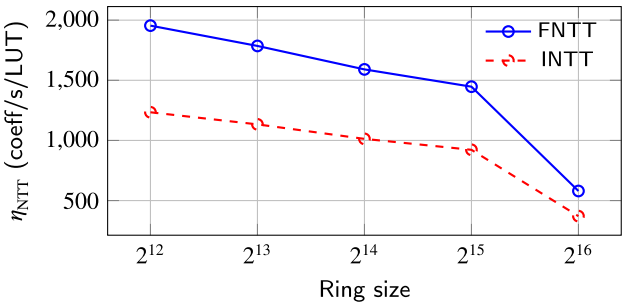

Fig. 4. Logic-normalized coefficient throughput (η_{NTT}) versus ring size for FNTT and INTT.

from microseconds to seconds. The defined metric quantifies the number of polynomial coefficients processed per second per unit of logic resource, thereby capturing both performance and hardware efficiency at the same time.

$$\eta_{NTT} = \frac{10^6 \cdot n}{L_{\mu s} \cdot A_{\text{logic}}} \tag{8}$$

Fig. 4 shows the logic-normalized coefficient throughput of the proposed architecture for both FNTT and INTT across different ring sizes. At smaller ring sizes, the design achieves higher efficiency, with η_{NTT} of approximately 1.95×10^3 and 1.23×10^3 coeff/s/LUT for FNTT and INTT, respectively, at 2^{12}. As the ring size increases, η_{NTT} gradually decreases, reaching approximately 5.8×10^2 coeff/s/LUT for FNTT and 3.7×10^2 coeff/s/LUT for INTT at 2^{16}. This reduction is primarily attributed to the sequential, area-optimized scheduling of the datapath, where transform latency grows more rapidly with ring size than the corresponding increase in logic resources.

Note that across all configurations, FNTT consistently exhibits higher efficiency than INTT, which is expected due to the additional inverse-mode overhead, including bit-reversal address generation and the final INTT scaling operation. Despite this overhead, the architecture maintains stable scalability characteristics, demonstrating predictable efficiency degradation as the transform size increases. These results confirm that the proposed design effectively balances logic utilization and performance, making it suitable for scalable NTT-based cryptographic accelerators operating under strict area constraints.

4.3. Comparison with state-of-the-art

Before presenting the comparison, it is important to clarify the design objective of this work relative to the state of the art. Most existing NTT accelerators [14,22–24,26] are throughput-optimized, employing multiple processing elements, deep pipelining, and parallelism to achieve high operating frequencies. The proposed architecture, instead, targets the area-minimization end of the design space, accepting higher latency in exchange for substantially reduced hardware costs. This positioning is intentional and directly motivated by the requirements of area-constrained HE platforms identified in Section 1.3. Therefore, the presented comparisons should be interpreted in this context.

The comparison to state-of-the-art NTT designs is shown in Table 3. To make a reasonable comparison of hardware resource usage with existing NTT designs, we have calculated the equivalent Slices (EqS) metric using the method described in [39].

Comparing hardware area in equivalent slices:. For a ring size of 2^{12} on a Virtex-7 FPGA, the proposed NTT accelerator achieves significantly lower hardware costs compared to prior configurable designs. Specifically, it is 1.85× ($\frac{30.0K}{16.2K}$) and 3.28× ($\frac{53.2K}{16.2K}$) more area efficient in terms of EqS compared to the compile-time configurable architectures reported in [26] and [27], respectively. These improvements stem from the proposed architecture's compact datapath organization and reduced control and memory overhead, which together minimize logic utilization without sacrificing functional completeness. Compared to [23], our Virtex-7 implementation achieves 9× lower equivalent slice utilization than the Zynq-7000 (Artix-7-based) design, reducing from 147.0K to 16.2K EqS. Although both devices are fabricated on 28nm technology, Virtex-7 targets high-performance designs with greater routing resources than the area-constrained Artix-7, making our area reduction even more notable.

For a ring size of 2^{15} on the same Virtex-7 platform, the proposed design maintains its area advantage. Compared to the dedicated NTT accelerator in [19], which employs extensive parallelism for throughput optimization, the proposed architecture reduces equivalent area by approximately 6.01×. Similarly, relative to the run-time configurable ReMCA architecture [35], the proposed design achieves a 2.57× reduction in EqS. These results highlight the effectiveness of the proposed sequential and memory-efficient scheduling strategy when area scalability is prioritized. Compared to [22], our design achieves 1.12× lower equivalent slice utilization for $n = 2^{15}$, reducing from 62.5K to 55.7K EqS on the same Virtex-7 device family.

Similarly, for the largest evaluated ring size of 2^{16}, the proposed NTT architecture is implemented on the same UltraScale ZU9EG device as the recent dedicated FNTT design in [14]. Under identical hardware conditions, the proposed design achieves a 2.34× reduction in equivalent slices, demonstrating that unified support for both forward and inverse transforms can be achieved without incurring excessive area overhead. Our ZU9EG-based implementation achieves 1.62× and 1.65× lower equivalent slice utilization compared to the Virtex-7-based [22] and ZCU102-based [24] designs, reducing from 82.2K and 83.9K to 50.7K EqS, respectively. Although the implementations target different FPGA platforms, the normalized EqS metric, computed via the formula given in the Table 3 footnote, ensures a fair and accurate cross-platform area comparison.

Comparing clock cycles and latency. In terms of clock cycles and absolute latency, existing NTT accelerators [14,19,22–24,26,27,35] generally exhibit lower execution times. This is expected, as these designs primarily target high-throughput operation through aggressive parallelism, deep pipelining, and higher operating frequencies. In contrast, the proposed architecture is explicitly optimized for area efficiency, resulting in increased cycle counts and latency. This reflects the fundamental trade-off between hardware cost and computational speed inherent in NTT accelerator design.

Comparing in terms of ATP. To further quantify this trade-off, Table 3 also reports the area-time product (ATP), defined as EqS×Latency. Under this metric, the proposed design over $n = 2^{15}$ with an identical Virtex-7 device achieves a 2.55× improvement compared to the high-parallelism NTT architecture of [19], indicating that substantial reductions in hardware costs can compensate for higher latency. However, designs such as [14,22–24,26,27,35] achieve lower ATP values due to their significantly higher operating frequencies and reduced cycle counts, which favor performance-oriented optimization.

Overall, the proposed NTT accelerator occupies a distinct design point in the NTT hardware design space, achieving the lowest equivalent slice utilization across all compared designs. While prior works prioritize throughput maximization, our architecture delivers superior area efficiency, with reductions of up to 9× over Zynq-7000-based [23], 1.62× over Virtex-7-based [22], and 1.65× over ZCU102-based [24] implementations for equivalent polynomial sizes. These gains are achieved while maintaining unified FNTT/INTT support, compile-time configurability, and large-modulus ($\log_2 q = 64$) operation, making the proposed design particularly suitable for resource-constrained cryptographic processors and post-quantum accelerators where hardware footprint and scalability are critical. The area efficiency demonstrated above results from two complementary factors: (i) the area–delay efficiency of the proposed integer multiplier, established in Section 4.1.1, and (ii) the structurally aware assignment of three dedicated multipliers that avoids

Table 3
Comparisons with existing state-of-the-art NTT hardware designs. The ✓ indicates where our design shows higher performance. The **CTC** and **RTC** show the features regarding configurability, which means compile-time-configurable and run-time-configurable, respectively. The **UFD** means unified designs supporting FNTT and INTT operations.

Ref.	Device	n	$\log_2 q$	LUTs	DSPs	BRAMs	Eqv Slices (EqS)[a]	Freq. (MHz)	Clock Cycles	Latency (μs)	ATP (EqS×Latency)	Design Features		
												CTC	RTC	UFD
[26]	VX485T	2^{12}	32	14.0K	80	79	30.0K	250	–	12	0.36	✓	✗	✓
[27]	VX690T	2^{12}	60	22.0K	248	96	53.2K	125	3276	26	1.38	✓	✗	✓
[23]	Zynq-7000	2^{12}	60	110.0K	768	176	147.0K	150	986	6.57	0.96	✗	✗	✗
[19]	VX690T	2^{15}	32	219.0K	768	869	335.3K	250	521,725	2087	699.77	✗	✗	✓
[35]	VX1140T	2^{15}	32	46.5K	400	392	143.6K	250	61,440	245	35.18	✗	✓	✓
[22]	Virtex-7	2^{15}	64	43.6K	361	63	62.5K	212	166	0.78	0.05	✓	✗	✓
[14]	ZU9EG	2^{16}	60	149K	564	616.5	119.1K	200	536,832	2684	319.66	✗	✗	✗
[24]	ZCU102	2^{16}	52	156.0K	896	160	83.9K	265	16426	61.98	5.20	✓	✓	✗
[22]	Virtex-7	2^{16}	64	47.3K	399	127	82.2K	196	331	1.68	0.13	✓	✗	✓
Ours	VX690T	2^{12}	64	4.2K	97	22.5	16.2K (✓)	100	49,190	491	7.95	✓	✗	✓
	VX690T	2^{15}	64	4.6K	97	192	55.7K (✓)	100	491,567	4915	273.76			
	ZU9EG	2^{16}	64	8.9K	97	384	50.7K (✓)	100	1,048,626	10,486	531.64			

[a] Eqv Slices (EqS) = # Slices + Eqv DSP + Eqv BRAM. In which, # Slice ≈ $\frac{\#LUT}{4}$ (for 7 series) or $\frac{\#LUT}{8}$ (for UltraScale), 1 DSP ≈ 102.4 Slices (for 7 series) or 51.2 Slices (for UltraScale), 36KB BRAM ≈ 232.4 Slices (for 7 series) or 116.2 Slices (for UltraScale) [39].

Table 4
NTT implementation results for ring size 2^{12}. The single shared-multiplier variants are derived from Fig. 1 by collapsing the three **MM** blocks into one shared instance connected via 3×1 input multiplexers on each operand port. Clock cycles remain identical across all variants, as all multipliers are fully combinational; BRAMs also remain identical; frequency differences arise solely from critical-path delay because of additional routing multiplexers.

Configuration	Slices	LUTs	FFs	DSPs	EqS	Freq. (MHz)	Clock Cycles	Latency (μs)	ATP (EqS×Latency)
Case 1: Three dedicated multipliers									
3× Schoolbook	4605	16339	3200	79	47.0K	39	49,190	1261	59.26
3× Karatsuba	4311	14380	3050	79	44.0K	31	49,190	1587	69.82
3× Direct (with * in HDL)	1062	3286	1850	127	17.5K	40	49,190	1230	21.52
3× Proposed[†]	1446	4285	2210	97	16.2K	100	49,190	491	7.95
Case 2: One shared multiplier + one 3×1 routing multiplexer at each input operand of the butterfly									
1× Schoolbook	1922	6555	1450	27	17.0K	25	49,190	1968	33.45
1× Karatsuba	1824	5902	1380	27	16.0K	19	49,190	2589	41.42
1× Direct (with * in HDL)	741	2204	1100	43	10.5K	25	49,190	1968	20.66
1× Proposed	869	2537	1580	33	10.5K	63	49,190	781	8.20

the multiplexing and control overhead incurred by shared-datapath designs [8,22,24]. Together, these factors validate the design motivation stated in Section 1.3: that assigning multipliers to structurally distinct computation paths, combined with an area–delay-efficient multiplier implementation, yields a lower overall NTT hardware area compared to prior configurable designs employing multipliers within shared datapaths.

4.4. Discussions over results and comparisons

To validate the structural multiplier assignment strategy and quantify the contribution of the proposed modular multiplier to overall NTT efficiency, Table 4 presents implementation results at $n = 2^{12}$ for the following two cases, with each case discussing four implementations:

- **Case 1:** It substitutes three dedicated multiplier instances of the Schoolbook, Direct (with * in HDL), and Recursive Karatsuba multipliers from Table 1 into Fig. 1 to evaluate their area-time trade-offs.
- **Case 2:** It uses an instance of a single shared-multiplier variant for each multiplier type (i.e., Schoolbook, Direct (with * in HDL), and Recursive Karatsuba), where the three MM blocks in Fig. 1 are collapsed into one instance with 3×1 input multiplexers on each operand port to evaluate their area-time trade-offs.

Table 4 provides experimental evidence supporting the architectural trade-offs of formulated case 1 and case 2. Before describing the results, it is essential to note that since all implemented multipliers in Table 1 are fully combinational, clock cycle counts remain identical at 49,190 across all variants in both cases. Differences in computation time or latency arise solely from the achievable operating frequency, which is directly determined by the critical-path delay of the implementation. Therefore, the results show that the proposed modular multiplier consistently achieves the best overall ATP among all evaluated configurations, regardless of whether dedicated or shared multiplier assignment is employed.

For the three dedicated multiplier configurations in case 1, the proposed design achieves an ATP of only 7.95, which is substantially lower than Schoolbook (59.26), Recursive Karatsuba (69.82), and the direct HDL multiplication implementation (21.52). Although the proposed architecture utilizes three dedicated multiplier instances, the reduced critical-path delay of the proposed multiplier enables operation at 100 MHz, which is significantly higher than all other implementations. This frequency improvement compensates for the additional multiplier replication and results in the best ATP across all evaluated designs.

Case 2 further evaluates the effect of collapsing the three multiplier blocks into a single shared multiplier connected through operand multiplexers. As expected, the shared-multiplier variants reduce area consumption because only one multiplier instance is retained. However, the introduced routing and multiplexing overhead significantly increases the critical-path delay and lowers the achievable operating frequency. For example, the proposed shared-multiplier configuration reduces the equivalent slices from 16.2K to 10.5K, but the frequency decreases from 100 MHz to 63 MHz, increasing latency from 491 μs to 781 μs and slightly degrading ATP from 7.95 to 8.20. Similar ATP degradation trends are also observed for Schoolbook and Karatsuba implementations.

These results experimentally confirm that simply minimizing the multiplier count does not necessarily improve overall efficiency. Although shared datapaths reduce area, the additional control, routing, and multiplexing overhead negatively impact timing performance. In contrast, the proposed dedicated-multiplier assignment eliminates routing multiplexers from the critical butterfly datapath and allows each arithmetic stage to operate independently, resulting in a substantially higher operating frequency and lower latency.

At the same time, Table 4 also clarifies the intended design objective of the proposed architecture. The proposed NTT accelerator is not optimized for maximum throughput; instead, it explicitly targets FPGA platforms where hardware area is the primary design constraint. This design choice results in a higher ATP compared with aggressively parallel state-of-the-art architectures such as ReMCA, which employ substantially larger hardware resources to minimize latency. Therefore, the proposed architecture should be interpreted as an area-efficient and structurally compact NTT solution rather than a throughput-optimized design. Within this design space, the proposed multiplier and dedicated structural assignment strategy provide the most favorable balance between area efficiency and timing performance.

5. Conclusions

This paper has presented a compile-time configurable NTT accelerator that supports polynomial degrees from 2^{12} to 2^{16} and modulus sizes up to 64 bits using a lightweight and efficient butterfly datapath. In contrast to conventional unified designs, the proposed architecture employs three dedicated modular multipliers—one each for the CT butterfly, GS butterfly, and inverse-scaling computations—together with lightweight routing multiplexers and arithmetic units. This structurally aware organization reduces control complexity, routing overhead, and datapath congestion, leading to more efficient hardware utilization.

Furthermore, an area-delay-efficient modular multiplication architecture has been implemented by integrating a novel integer polynomial multiplier with an optimized Montgomery-based modular reduction unit. The use of parallel partial-product generation, accumulation, and shifting techniques in the integer multiplier, combined with operand digitization in the Montgomery reduction architecture, effectively reduces logic and routing delays while supporting large-modulus arithmetic required in practical homomorphic encryption systems.

Implementation results on an Artix-7 FPGA device and comprehensive comparisons with state-of-the-art designs demonstrate that the proposed architecture achieves substantially lower area than existing configurable NTT accelerators, with only modest clock-cycle overhead. These results confirm the effectiveness of the proposed techniques for realizing area-efficient NTT hardware in resource-constrained cryptographic applications.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the author(s) used Copilot for grammar checking and sentence restructuring. After using this tool, the author(s) reviewed and edited the content as needed and take full responsibility for the final version of the manuscript.

CRediT authorship contribution statement

Omar S. Sonbul: Writing – review & editing, Validation, Software, Methodology, Investigation, Conceptualization; **Muhammad Rashid:** Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Project administration, Methodology, Investigation, Conceptualization; **Amar Y. Jaffar:** Writing – review & editing, Validation, Resources, Data curation; **Mohammed J. Alghamdi:** Writing – review & editing, Validation; **Muhammad I. Masud:** Writing – review & editing, Resources, Investigation, Formal analysis; **Mohammed Aman:** Writing – review & editing, Resources, Funding acquisition.

Data availability

No data was used for the research described in the article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research was funded by Umm Al-Qura University, Saudi Arabia under grant number: 26UQU4320199GSSR01.

Acknowledgments

The authors extend their appreciation to Umm Al-Qura University, Saudi Arabia for funding this research work through grant number: 26UQU4320199GSSR01.

References

[1] J. Park, S. Kim, J. Kim, J.H. Ahn, Unlocking private computation at scale: the acceleration of homomorphic encryption, Computer 58 (12) (2025) 144–148. https://doi.org/10.1109/MC.2025.3613184
[2] J. Zhao, H. Yang, M. Hao, W. Zhang, H. He, D. Wang, HEngine: a high performance optimization framework on a GPU for homomorphic encryption, ACM Trans. Archit. Code Optim. 22 (2) (2025). https://doi.org/10.1145/3732942
[3] K. Munjal, R. Bhatia, A systematic review of homomorphic encryption and its contributions in healthcare industry, Complex Intell. Syst. 9 (4) (2023) 3759–3786.
[4] D.R. Chirra, The role of homomorphic encryption in protecting cloud-based financial transactions, Int. J. Adv. Eng. Technol. Innov. 1 (01) (2023) 452–472.
[5] F. Effendi, A. Chattopadhyay, Privacy-preserving graph-based machine learning with fully homomorphic encryption for collaborative anti-money laundering, in: International Conference on Security, Privacy, and Applied Cryptography Engineering, Springer, 2024, pp. 80–105.
[6] O.S. Sonbul, M. Rashid, M.I. Masud, M. Aman, A.Y. Jaffar, Deeply pipelined NTT accelerator with ping-pong memory and LUT-only barrett reduction for post-quantum cryptography, Electronics 15 (3) (2026) 513.
[7] S.-H. Liu, C.-Y. Kuo, Y.-N. Mo, T. Su, An area-efficient, conflict-free, and configurable architecture for accelerating NTT/INTT, IEEE Trans. Very Large Scale Integr. Syst. 32 (3) (2023) 519–529. https://doi.org/10.1109/TVLSI.2023.3336951.
[8] M. Imran, S. Khan, A. Khalid, C. Rafferty, Y.A. Shah, S. Pagliarini, M. Rashid, M. O'Neill, Evaluating NTT/INTT implementation styles for post-quantum cryptography, IEEE Embed. Syst. Lett. (2024) 1. https://doi.org/10.1109/LES.2024.3410516.
[9] X. Zhang, Y. Wei, M. Li, J. Tian, Z. Wang, HRCIM-NTT: an efficient compute-in-memory NTT accelerator with hybrid-redundant numbers, IEEE Trans. Circuits Syst. I: Regul. Pap. 72 (1) (2025) 214–227. https://doi.org/10.1109/TCSI.2024.3463184
[10] R. Bevin, A. Khalid, M. Imran, M. O'Neill, Accelerating CRYSTALS-Kyber and dilithium via a single montgomery reduction ISE on RISC-V, in: 2025 IEEE 38th International System-on-Chip Conference (SOCC), 2025, pp. 1–6. https://doi.org/10.1109/SOCC66126.2025.11235487
[11] O.S. Sonbul, M. Rashid, A.Y. Jaffar, Accelerating CRYSTALS-Kyber: high-speed NTT design with optimized pipelining and modular reduction, Electronics 14 (11) (2025) 2122.
[12] M. Imran, A. Khalid, C. Rafferty, M. Rashid, M. O'Neill, Evaluating area-time-power trade-offs in NTT accelerators for post-quantum cryptography, in: 2025 IEEE 38th International System-on-Chip Conference (SOCC), 2025, pp. 1–6. https://doi.org/10.1109/SOCC66126.2025.11235339
[13] J. Mo, A. Daftardar, J. Ah-Kiow, K. Guo, B. Bünz, S. Garg, B. Reagen, MTU: the multifunction tree unit for accelerating zero-knowledge proofs, in: Proceedings of the 14th International Workshop on Hardware and Architectural Support for Security and Privacy (HASP), 2025, pp. 1–9. https://doi.org/10.1145/3768725.3768737
[14] P. Duong-Ngoc, S. Kwon, D. Yoo, H. Lee, Area-efficient number theoretic transform architecture for homomorphic encryption, IEEE Trans. Circuits Syst. I: Regul. Pap. 70 (3) (2023) 1270–1283. https://doi.org/10.1109/TCSI.2022.3225208.
[15] N. Standards, FIPS 203: module-lattice-based key-encapsulation mechanism standard, 2024. Federal Information Processing Standard Publication 203, https://csrc.nist.gov/pubs/fips/203/final.
[16] H. Chen, K. Laine, R. Player, Simple encrypted arithmetic library - SEAL v2.1, 2017, (Cryptology ePrint Archive, Paper 2017/224). https://eprint.iacr.org/2017/224.
[17] M. Rashid, S. Khan, O.S. Sonbul, S.O. Hwang, A flexible and parallel hardware accelerator for forward and inverse number theoretic transform, IEEE Access 12 (2024) 181351–181361.

[18] G. Alsuhli, H. Saleh, M. Al-Qutayri, B. Mohammad, T. Stouraitis, Efficient NTT/INTT processor for FALCON post-quantum cryptography, J. Inf. Secur. Appl. 93 (2025) 104177. https://doi.org/10.1016/j.jisa.2025.104177

[19] E. Öztürk, Y. Doröz, E. Savaş, B. Sunar, A custom accelerator for homomorphic encryption applications, IEEE Trans. Comput. 66 (1) (2017) 3–16. https://doi.org/10.1109/TC.2016.2574340.

[20] S. Kurniawan, P. Duong-Ngoc, H. Lee, Configurable memory-based NTT architecture for homomorphic encryption, IEEE Trans. Circuits Syst. II: Express Br. 70 (10) (2023) 3942–3946. https://doi.org/10.1109/TCSII.2023.3289489.

[21] J. Huang, C. Kuo, S. Liu, T. Su, An area-efficient and configurable number theoretic transform accelerator for homomorphic encryption, Electronics 13 (17) (2024). https://doi.org/10.3390/electronics13173382.

[22] M.A. Zaman, M. Bilal Khan, W. Ahmad, A. Khalid, M. O'Neill, A lightweight and hardware-efficient NTT FPGA accelerator for FHE applications, IEEE Trans. Circuits Syst. II: Express Br. 73 (1) (2026) 78–82. https://doi.org/10.1109/TCSII.2025.3633181.

[23] J. Qiu, R. Huang, J. Shen, B. Lin, A unified and resource-efficient polynomial multiplication architecture for fully homomorphic encryption, J. King Saud Univ. Comput. Inf. Sci. (2026). https://doi.org/10.1007/s44443-026-00492-1

[24] F. Krieger, F. Hirner, A.C. Mert, S.S. Roy, A flexible hardware design tool for fast fourier and number-theoretic transformation architectures, 2025, (Cryptology ePrint Archive, Paper 2025/1407). https://eprint.iacr.org/2025/1407.

[25] C. Wang, M. Gao, SAM: a scalable accelerator for number theoretic transform using multi-dimensional decomposition, in: 2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD), 2023, pp. 1–9. https://doi.org/10.1109/ICCAD57390.2023.10323744

[26] Y. Su, B.-L. Yang, C. Yang, Z.-P. Yang, Y.-W. Liu, A highly unified reconfigurable multicore architecture to speed up NTT/INTT for homomorphic polynomial multiplication, IEEE Trans. Very Large Scale Integr. Syst. 30 (8) (2022) 993–1006. https://doi.org/10.1109/TVLSI.2022.3166355.

[27] A.C. Mert, E. Karabulut, E. Öztürk, E. Savaş, M. Becchi, A. Aysu, A flexible and scalable NTT hardware : applications from homomorphically encrypted deep learning to post-quantum cryptography, in: 2020 Design, Automation & Test in Europe Conference & Exhibition (DATE), 2020, pp. 346–351. https://doi.org/10.23919/DATE48585.2020.9116470.

[28] CoHA-NTT: a configurable hardware accelerator for NTT-based polynomial multiplication, Microprocess. Microsyst. 89 (2022) 104451. https://doi.org/10.1016/j.micpro.2022.104451.

[29] A.C. Mert, E. Öztürk, E. Savaş, FPGA implementation of a run-time configurable NTT-based polynomial multiplication hardware, Microprocess. Microsyst. 78 (2020) 103219. https://doi.org/10.1016/j.micpro.2020.103219

[30] A.C. Mert, E. Karabulut, E. Öztürk, E. Savaş, A. Aysu, An extensive study of flexible design methods for the number theoretic transform, IEEE Trans. Comput. 71 (11) (2022) 2829–2843. https://doi.org/10.1109/TC.2020.3017930.

[31] W. He, S. Kim, A survey on high-level synthesis approaches for number theoretic transform on FPGAs, in: 2025 IEEE 68th International Midwest Symposium on Circuits and Systems (MWSCAS), 2025, pp. 337–341. https://doi.org/10.1109/MWSCAS53549.2025.11244458

[32] National Institute of Standards and Technology, FIPS 204: module-lattice-based digital signature standard, 2024, Federal Information Processing Standard Publication 204, https://csrc.nist.gov/pubs/fips/204/final.

[33] National Institute of Standards and Technology, FIPS 205: stateless hash-based digital signature standard, 2024, Federal Information Processing Standard Publication 205, https://csrc.nist.gov/pubs/fips/205/final.

[34] F. Hirner, A.C. Mert, S.S. Roy, Proteus: a pipelined ntt architecture generator, 2023, (Cryptology ePrint Archive, Paper 2023/267). https://doi.org/10.1109/TVLSI.2024.3377366

[35] Y. Su, B.-L. Yang, C. Yang, S.-Y. Zhao, ReMCA: a reconfigurable multi-core architecture for full RNS variant of BFV homomorphic evaluation, IEEE Trans. Circuits Syst. I: Regul. Pap. 69 (7) (2022) 2857–2870. https://doi.org/10.1109/TCSI.2022.3163970.

[36] T. Hikino, W. Ohashi, H. Tomiyama, Fast 64-bit integer multipliers for FPGAs, in: 2025 22nd International SoC Design Conference (ISOCC), 2025, pp. 1–2. https://doi.org/10.1109/ISOCC66390.2025.11329911

[37] M. Imran, Z.U. Abideen, S. Pagliarini, An open-source library of large integer polynomial multipliers, in: 2021 24th International Symposium on Design and Diagnostics of Electronic Circuits & Systems (DDECS), 2021, pp. 145–150. https://doi.org/10.1109/DDECS52668.2021.9417065.

[38] M. Imran, Z.U. Abideen, S. Pagliarini, A versatile and flexible multiplier generator for large integer polynomials, J. Hardw. Syst. Secur. 7 (2) (2023) 55–71. https://doi.org/10.1007/s41635-023-00134-2

[39] D.-e.-S. Kundi, Y. Zhang, C. Wang, A. Khalid, M. O'Neill, W. Liu, Ultra high-speed polynomial multiplications for lattice-based cryptography on FPGAs, IEEE Trans. Emerg. Top. Comput. 10 (4) (2022) 1993–2005. https://doi.org/10.1109/TETC.2022.3144101.